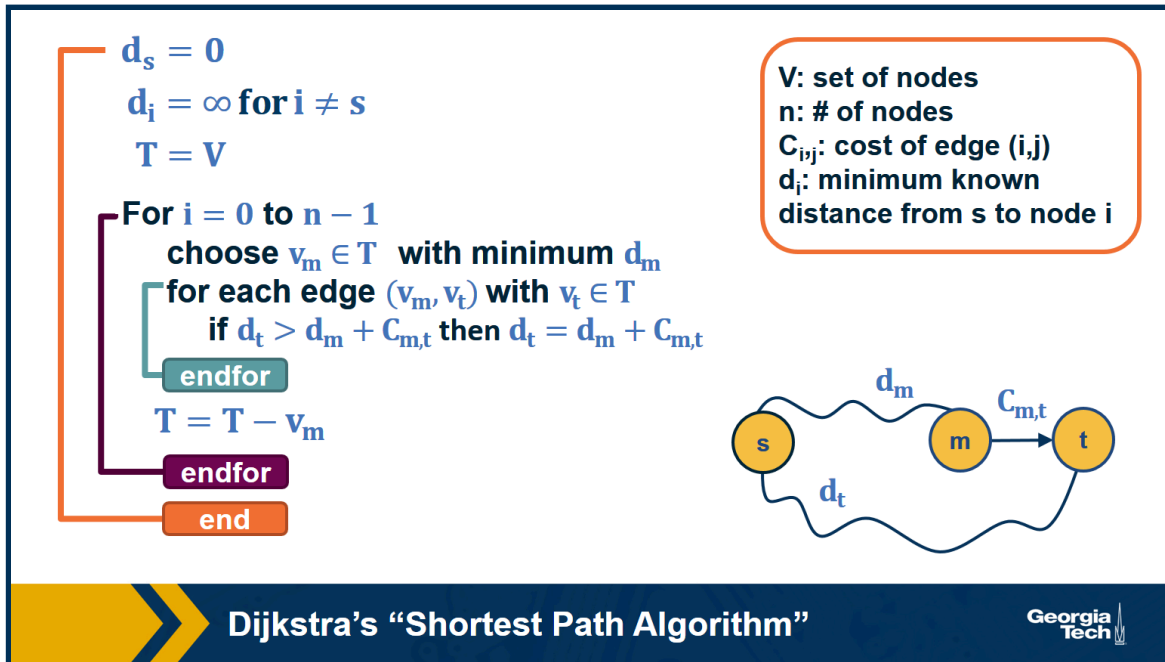


L2: Dijkstra's Shortest Path Algorithm

We are often interested in the shortest path (*or paths*) between a pair of nodes. Such paths represent the most efficient way to move in a network.

In unweighted networks, all edges have the same cost, and the shortest path from a node s to any other node in the same connected component (*or SCC for directed networks*) can be easily computed in linear time using a Breadth-First Search traversal from node s .



If the network is weighted, and the weight of each edge is its "**length**" or "**cost**", we can use Dijkstra's algorithm, showed above, to compute the shortest path from s to any other node. **Note that this algorithm is applicable only if the weights are positive.**

The key idea in the algorithm is that in each iteration we select the node m with the minimum known distance from s – that distance cannot get any shorter in subsequent iterations. We then update the minimum known distance to every neighbor of m that we have not explored yet, if the latter is larger than the distance from s to m plus the cost of the edge from m to t .

If the network is weighted and some weights are negative, then instead of Dijkstra's algorithm we can use the Bellman-Ford algorithm, which is a classic example of dynamic programming. The running-time of Bellman-Ford is $O(mn)$, where m is the number of edges and n is the number of nodes. On the other hand, the running time of Dijkstra's algorithm is $O(m + n \log n)$ if the latter is implemented with a Fibonacci heap (*to identify the node with the minimum distance from s in each iteration of the loop*).

Food for Thought

If you are not familiar with Fibonacci heaps, we suggest you review that data structure at:

https://en.wikipedia.org/wiki/Fibonacci_heap $\Rightarrow$ https://en.wikipedia.org/wiki/Fibonacci_heap